Chapitre 7

Algorithmes d'approximation.

On souhaite étudier des problèmes NP-difficiles en approximant la meilleure solution.

Exemple 1: Couverture par les sommets.

Instance: Un graphe $G = (S, A)$.
Solution: Un sous-ensemble $S' \subset S$ tel que $\forall (s,t) \in A,\ s \in S'$ ou $t \in S'$.
Fonction à minimiser : $|S'|$.

Solution :

Idée 1. Tant qu'il reste des arêtes, prendre un sommet de S, puis retirer toutes les arêtes adjacentes.
Variant de boucle : nombre d'arêtes; atteint 0. Invariant : si une arête (s,t) est retirée, $s \in S'$ ou $t \in S'$.
Cet algorithme n'est pas optimal.

Définition 2: λ-approximation.

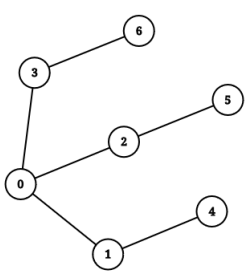
Pour un problème P minimisant la fonction f, on dit d'un algorithme qu'il s'agit d'une **λ-approximation** si pour tout instance, on a $\lambda f(s_{\text{OPT}}) \geq f(s_A)$ où s_{OPT} est optimale et s_A la solution de l'algorithme.
Dans ce cas, $f(s_A) \in [f(s_{\text{OPT}}), \lambda f(s_{\text{OPT}})]$; plus λ est proche de 1, plus la solution renvoyée est optimale.
On parle aussi d'approximation à facteur constant car λ ne dépend pas de l'instance considérée.

Preuve :

Pour l'exemple précédent, on n'a pas de λ-approximation. Supposons qu'on en ait une, et considérons le graphe E_k, en étoile à $k = \lfloor \lambda \rfloor + 1$ branches. Sur E_k, l'algorithme glouton peut renvoyer tous les sommets de degré 1, on a alors $f(s_{\text{OPT}}) = 1$ et $f(s_A) = \lfloor \lambda \rfloor + 1$. On aurait alors $\lambda \geq \lfloor \lambda \rfloor + 1$, absurde.

Exemple 3: Modification 1.

On modifie maintenant l'algorithme pour que s soit choisi comme sommet de plus haut degré.
Il n'est pas optimal, par exemple sur le graphe suivant :



On n'a pas de λ-approximation non-plus, mais la preuve est plus complexe (exemple suivant).

Exemple 4: Pas de λ-approximation.

On prend un graphe avec k couches de sommets. La couche i contient $\left\lfloor \frac{k}{i} \right\rfloor$ sommets. Chaque sommet de la couche i est relié à i sommets de la couche 1. Il est le seul sommet de cette couche à être relié à ces sommets.
Pour n'importe quel k, il existe une solution de taille k. Il suffit de prendre tous les sommets de la première couche. Si un sommet s appartient à la couche $i > 1$, son degré est exactement i. Les sommets de la couche 1 sont reliés à au plus $k-1$ sommets (au plus un sommet de chaque autre couche). L'algorithme glouton choisit donc le sommet de la couche k. Les sommets de la couche 1 sont alors de degré au plus $k-2$. Finalement, on ajoute tous les sommets n'appartenant pas à la première couche. La taille de la solution est en $\sum_{i=2}^{k} \left\lfloor \frac{k}{i} \right\rfloor = \Theta(k \log(k))$.
S'il s'agissait d'une λ-approximation, on aurait $\frac{f(s_A)}{f(s_{\text{OPT}})} \leq \lambda$ or $\frac{f(s_A)}{f(s_{\text{OPT}})} \geq \sum_{i=1}^{k} \frac{1}{i} - 2 + \frac{1}{k}$.
On aurait alors $\lambda \xrightarrow[k \to +\infty]{} +\infty$, ce qui est absurde.

Exemple 5: Modification 2.

On fait une dernière modification : au lieu de choisir un sommet s, on choisit une arête (s,t) et on ajoute à la fois s et t dans S'. Ce n'est pas optimal, sur le graphe étoile, il renvoie toujours une solution de taille $2k$. Sur le graphe de l'exemple précédent aussi. Cet algorithme glouton est une 2-approximation.

Solution :

On remarque que lorsqu'on ajoute s et t à S', on est certain qu'au moins l'un des deux appartient à la solution optimale. Ainsi, si l'algorithme glouton réalise k itérations, toutes les solutions contiennent au moins k sommets. L'algorithme glouton renvoie une solution à $2k$ sommets. On a $f(s_{\text{OPT}}) \geq k$, donc $2f(s_{\text{OPT}}) \geq 2k = f(s_A)$.
Il s'agit bien d'une 2-approximation.

Exemple 6: Programmation de tâches.

Instance. Un nombre p de processeurs, une liste de durées (d_i).
Solution. Une affectation de chaque tâche à un processeur.
Fonction à minimiser : temps de fin d'exécution du dernier processeur.
Le problème de décision associé est `NP`-difficile, on le montre en exhibant une réduction de 2-Partition.

Solution :

Idée 1. Pour chaque tâche : l'affecter au processeur le moins utilisé. On peut montrer que si $p = 2$, il s'agit d'une $\frac{3}{2}$-approximation, et que cette borne est atteinte. Atteint par exemple pour $(1,1,2)$.
Amélioration. On peut trier toutes les taches par durée décroissante. On obtient une $\frac{7}{6}$-approximation dans le cas à 2 processeurs. Atteint par exemple pour (3,3,2,2,2)

Proposition 7: Preuve d'inapproximabilité.

Le problème du Voyageur de commerce ne possède pas de λ-approximation en temps polynomial, sauf si $\mathtt{P} = \mathtt{NP}$.

Preuve :

On raisonne par réduction comme pour la `NP`-complétude. On suppose qu'il existe un algorithme de λ-approximation en temps polynomial pour le Voyageur de Commerce. On considère une instance $G = (S, A)$ de cycle hamiltonien. On cherche à construire une instance de Voyageur de Commerce sur laquelle on pourra appliquer notre λ-approximation. On aimerait que le résultat de celle-ci nous indique si oui ou non il y avait un cycle hamiltonien dans G. On veut compléter le graphe avec des arêtes de poids $f(\lambda)$, celles déjà présentes ayant un poids de 1. Supposons que G contienne un cycle hamiltonien, la solution optimale aura un poids de n. Puisqu'on utilise une λ-approximation, on a $f(s_A) \leq \lambda n$. Si on prend $f(\lambda) = \lambda n$, alors si G admet un cycle hamiltonien ssi $f(s_A) \leq \lambda n$. Si il existe un tel algorithme, on a alors un algorithme polynomial pour répondre à Cycle-Hamiltonien. Or Cycle-Hamiltonien est `NP`-difficile, alors $\mathtt{P} = \mathtt{NP}$.

Proposition 8

Dans le cas où les poids respectent l'inégalité triangulaire, un algorithme d'approximation :

Entrées : $G = (S, A)$ un graphe pondéré par p
$T \leftarrow$ arbre couvrant minimal de G.
Enraciner cet arbre.
Parcourir cet arbre en notant chaque sommet visité.
Éliminer les doublons dans ce parcours pour obtenir un cycle hamiltonien.

L'algorithme proposé renvoie une 2-approximation.

Preuve :

On a $p(T) \leq f(s_{\text{OPT}})$. En effet, si on prend le cycle hamiltonien et si on lui retire une arête, on obtient un arbre couvrant de poids au moins $p(T)$. Dans le parcours, chaque arête est comptée 2 fois. Le poids du chemin est donc $2p(T)$. Le chemin finalement obtenu a un poids majoré par $2f(s_{\text{OPT}})$. Il s'agit donc bien d'une 2-approximation. Cette borne n'est jamais atteinte, si tous les poids sont non nuls, la première inégalité est stricte : $p(T) < f(s_{\text{OPT}})$, ce qui empêche la borne d'être atteinte.

Définition 9: Approximation à facteur inconstant.

Planification de tâches peut être approximé à un facteur $1 + \varepsilon$ en temps polynomial lorsque $p = 2$.

Preuve :

On note $L = \max\{d_{max},\ \frac{1}{2}\sum_{i=1}^{n} d_i\}$.

$$\frac{1}{2}\sum d_i \leq f(s_{\text{OPT}}) \Longrightarrow d_{max} \leq f(s_{\text{OPT}}) \Longrightarrow L \leq f(s_{\text{OPT}}).$$

On partitionnera les tâches :
— Les petites de durées inférieures à εL.
— Les grosses de durées supérieures à εL.

Définition 10

— Affecter les grosses tâches de manière optimale.
— Affecter les petites tâches de manière gloutone.

Proposition 11: Coût de l'algorithme.

Combien y-a-t-il de grosses tâches ?

Preuve :

Chaque grosse tâche a une durée supérieure à $\frac{\varepsilon}{2}\sum d_i$.
Si on a n grosse tâches, leurs poids total est d'au moins $\frac{n\varepsilon}{2}\sum d_i \leq \sum d_i$ donc $n \leq \frac{2}{\varepsilon}$.
Le nombre de grosses tâches est borné par $\frac{2}{\varepsilon}$. Alors ε ne dépend pas de l'instance. On peut donc le considérer comme constant.
Énumérer toutes les affectations de grosses tâches n'est donc pas réalisé en temps exponentiel mais bien en temps polynomial. L'algorithme s'exécute bien en temps polynomial. Montrons qu'il s'agit bien d'une $1 + \varepsilon$-approximation.
— L'affectation des petites tâches n'a pas modifié le temps d'exécution du plus lent.

On considère l'instance I^* restreinte avec grosses tâches. L'algorithme est optimale sur I^* :

$$f(s_{\text{OPT}}(I)) \geq f(s_{\text{OPT}}(I^*))$$

On a $f(s_{\text{OPT}}(I*)) = f(s_A(I))$, et $f(s_{\text{OPT}}(I)) =\geq f_{s_A(I^*)}$. Par double inégalité, $f(s_{\text{OPT}}) = f(s_A)$.
— Sinon, il existe d_k, une petite tâche, affectée au processeur le plus lent. Lorsqu'elle a été affectée, il s'agissait du processeur le moins occupé, son temps d'occupation était inférieur à $\frac{1}{2}\sum d_i$.
Après affectation, on obtient $\frac{1}{2}\sum d_i + d_k = f(s_A)$ au plus. Alors $\frac{1}{2}\sum d_i \leq L$, $d_k \leq \varepsilon L$ puis $f(s_A) \leq (1+\varepsilon)L$.
Notre algorithme s'exécute en $O(2^{\frac{1}{\varepsilon}} + P(n))$ où P est un polynôme. ε est en réalité un paramètre même s'il n'est pas un élément de l'instance. La complexité de l'algorithme est exponentielle en $\frac{1}{\varepsilon}$. Notre O cache une constante très grande lorsque ε est petit. On parle de méthode d'approximation polynomiale pour les algorithmes qui prennent en paramètre ε et une instance et renvoient une $(1+\varepsilon)$-approximation en temps polynomial en la taille de l'instance. On parle de méthode d'approximation complètement polynomiale lorsque l'algorithme s'exécute aussi en temps polynomial en $\frac{1}{\varepsilon}$.

Définition 12: Coloration de graphe.

Instance: Un graphe G. **Solution:** Une coloration des sommets de G telle que $\forall (s,t) \in A,\ c(s) \neq c(t)$.
Fonction à minimiser : Le nombre de valeurs que prend c.

---

tant que il existe un sommet non coloré **faire**
  En choisir un s.
  Colorer s avec la plus petite couleur non utilisée par ses voisins.
fin

Preuve :

Dans le cas où tous les voisins d'un sommet s sont de couleur différente, on utilise $d(s)+1$ couleurs pour colorer s et ses voisins. Pour colorer tout le graphe, on utilise donc au plus $d_{max}+1$ couleurs. Si G n'est pas 2-colorable, on peut utiliser l'algorithme glouton pour avoir une $\frac{d_{max}+1}{3}$-approximation :

$$f(s_{\text{OPT}}) \geq 3 \Longrightarrow f(s_A) \leq d_{max}+1 \Longrightarrow f(s_A) = \frac{1}{3}f(s_A) \leq \frac{d_{max}+1}{3}f(s_{\text{OPT}}).$$

S'il est 2-colorable, on trouve une coloration en temps polynomial en réalisant un parcours du graphe, au cours duquel on colorie les sommet en alternant les couleurs. Si le graphe n'est pas 2-colorable, on s'en rend compte au cours du parcours. On peut donc essayer de colorer le graphe avec deux couleurs puis d'utiliser l'algorithme glouton en cas d'échec.

Définition 13: 3-coloration.

tant que il existe un sommet s de degré supérieur à $\sqrt{n}$ **faire**
| Colorer ses voisins avec deux nouvelles couleurs.
fin
Colorer les sommets restants avec l'algorithme glouton.

Preuve :

À chaque itération, on colore au moins $\lceil \sqrt{n} \rceil$ sommets, on fait donc au plus $\sqrt{n}$ itérations.
Le graphe des voisins de s est 2-colorable car si l'on rajoute s, il est 3-colorable, or, s est rélié à tous les autres. Ceux-ci n'ont donc que deux couleurs possibles.
À la fin de la boucle, on a utilisé $2\sqrt{n}$ couleurs au plus. L'algorithme glouton est exécuté sur un graphe de degré maximal inférieur à $\sqrt{n}$, il utilise donc au plus $\sqrt{n}+1$ couleurs.
À la fin de la boucle, on a attribué $2\sqrt{n}$ couleurs au plus. L'algorithme glouton est exécuté sur un graphe de degré maximal inférieur à $\sqrt{n}$. Il utilise donc au plus $\sqrt{n+1}$ couleurs. L'algorithme colorie avec au plus $3\sqrt{n}+1$ en tout.
Cette borne est rarement atteinte, car la première inégalité découle du fait d'avoir réalisé beaucoup d'itérations, tandis que la deuxième vient du fait d'en avoir fait peu.